

API Security Risk Analysis Report

Cyber Security Internship — Future Interns

Intern: Lalas Maruthi | CIN ID: FIT/AUG26/CS10164

Target: reqres.in (public demo API, explicitly published for testing)

Date: 31-08-2026

1. Why this target

I used reqres.in for this analysis because it's a demo API that's meant to be tested against — there's no real user data behind it, so I could poke at it freely and focus on learning the methodology rather than worrying about causing harm or working somewhere I wasn't authorized to be.

2. How I tested it

I used Postman to send requests manually rather than just scripting everything blind — I wanted to actually look at each response and think about what it meant before moving to the next check.

- Mapped out what endpoints exist and what each one returns
- Checked whether protected-looking endpoints actually enforce authentication
- Tried swapping resource IDs to see if I could pull data that wasn't "mine" (IDOR check)
- Sent a burst of repeated requests to see if there's any rate limiting in place
- Sent deliberately malformed input to see how the API reacts

3. What I found

Nothing here is a real incident since this is a demo service, but each finding maps to something I'd absolutely check on a real, production API.

ID	Finding	Risk	Category
A-01	No rate-limiting on repeated requests	Medium	Resource abuse
A-02	Sequential, predictable resource IDs	Medium	Broken object-level auth
A-03	Verbose error responses on bad input	Low	Information disclosure
A-04	No authentication required to read data	Medium	Access control (expected on this demo)

4. Going through each finding

A-01: No rate limiting

I fired off fifty requests back to back and every single one came back 200 OK — nothing throttled me. On a real system, that means someone could brute-force a login endpoint or scrape data with basically no resistance.

- Add per-IP or per-key rate limits and return 429 once they're hit
- Put stricter limits specifically on login/auth endpoints

A-02: Predictable IDs

Resources here are just numbered 1, 2, 3... which means if authorization isn't checked properly on a real system, someone could just increment the number in the URL and see other people's data. That's called IDOR, and it's one of the most common — and most preventable — API bugs out there.

- Use non-guessable identifiers (UUIDs) instead of sequential numbers for anything sensitive
- Check authorization on every single request, not just at login time

A-03: Verbose errors

Sending bad input got me back an error message with more internal detail than a real system should ever expose to a client — that kind of information makes an attacker's job easier by handing them a head start.

- Return generic errors to the client, keep the detailed logs server-side only

A-04: No auth required

Expected here since it's a public demo, but worth stating plainly: on anything real, every endpoint that returns user or business data needs to check for a valid token on every request — not just trust that a login happened at some point earlier.

- Require a valid bearer token/API key on every non-public endpoint
- Only return the fields the client actually needs — don't over-share data by default

5. What I'd fix first if this were live

- **Immediately: authentication and authorization on every data-returning endpoint**
- Soon after: rate limiting and switching away from sequential IDs
- Ongoing: generic error handling and scheduled re-testing

6. What this task taught me

Doing this by hand in Postman instead of just running an automated scanner made the IDOR issue click for me in a way reading about it never did — actually watching User 2's data come back when I typed in a different number was a much clearer lesson than any explanation. I plan to run this exact same checklist against my next assigned target as the real, final version of this task.